

Техническое Задание и Архитектура Системы

Полный инженерный спецификат автономного мультимодального робота-агента "AURA-Bot"

ОСНОВНАЯ МИКРОСХЕМА

ESP32-S3 N16R8 (16M Flash, 8M Octal PSRAM)

СЕРВЕР ЛОКАЛЬНОГО ИИ

MSI GP66 (RTX 3070/3080)

ДИСПЛЕЙ И АНИМАЦИЯ

GMT130-v1.0 IPS 240x240 (ST7789)

ПАМЯТЬ И RAG

Obsidian Vault + ChromaDB (Vector DB)

ПРОТОКОЛ СВЯЗИ

WebSocket / WebRTC Stream over Wi-Fi

1. Аппаратная Архитектура и Распределение GPIO (Pinout)

Для предотвращения аппаратных конфликтов с встроенной PSRAM (которая на N16R8 работает в Octal-режиме и занимает GPIO 33-37), сигнальными пинами камеры (GPIO 4-21) и boot-страппинг пинами (GPIO 0, 45, 46), распиновка строго зафиксирована.

1.1. Фиксированная Таблица Подключений ESP32-S3

Модуль	Выход Модуля	GPIO ESP32-S3	Тип / Примечание
INMP441 (I2S Микрофон)	VDD	3.3V	Питание шины 3.3 В
	GND / L/R	GND	Левый канал (GND)
	SD (Data Out)	GPIO 42	Сохранено из текущей рабочей конфигурации

Модуль	Вывод Модуля	GPIO ESP32-S3	Тип / Примечание
MAX98357A (I2S Усилитель)	WS (Word Select)	GPIO 1	Сохранено из текущей рабочей конфигурации
	SCK (Bit Clock)	GPIO 2	Сохранено из текущей рабочей конфигурации
	Стабильность проверена на рабочей прошивке XiaoZhi.		
	LRC (WS)	GPIO 14	Шина I2S TX (Воспроизведение)
	BCLK (SCK)	GPIO 13	Шина I2S TX (Bit Clock)
GMT130-v1.0 IPS (ST7789 240x240)	DIN (Data In)	GPIO 12	Шина I2S TX (Data Line)
	GAIN / SD_MODE	GND / 3.3V	Усиление по умолчанию (12dB gain)
	VIN / GND	5V / GND	ВАЖНО: Питание 5V от USB для громкости звука
	VCC / GND	3.3V / GND	Питание логики дисплея
	SCL (CLK)	GPIO 39	SPI Clock (Hardware SPI2)
EMAX ES08MA II (Аналоговые Серво)	SDA (MOSI)	GPIO 38	SPI Master Out (Hardware SPI2)
	RES (Reset)	GPIO 40	Hardware Reset дисплея
	DC (Data/Command)	GPIO 41	Переключение Режим Данные/Команда
	BLK (Backlight)	GPIO 48	ШИМ Управление яркостью подсветки
EMAX ES08MA II (Аналоговые Серво)	Servo 1 (Yaw - Поворот)	GPIO 47	ШИМ Управление (LEDC Channel 0, 50Hz)
	Servo 2 (Pitch - Кивок)	GPIO 21	ШИМ Управление (LEDC Channel 1, 50Hz)

Модуль	Вывод Модуля	GPIO ESP32-S3	Тип / Примечание
OV2640 / OV5640 Camera	DVP Data & Clock	GPIO 4-11, 15-18	Стандартная разводка ESP32-S3 CAM плат
	PWDN / RESET	GPIO -1 / 38 (через экспандер или параллельно)	Внутреннее управление камерой

1.2. Схема Питания и Фильтрации Наводка/Шума (Использование Доп. Компонентов)

Схема Подключения Питания и Устранения Перепомех:

- **Проблема импульсных токов сервоприводов:** Сервоприводы EMAX ES08MA II в моменты пуска потребляют до 800мА. Это вызывает просадку напряжения на ESP32-S3 и перегружает внутренний LDO-регулятор, что приводит к **Brownout Detector Reset** или сильным шумам в микрофоне INMP441.
- **Решение с использованием имеющихся деталей:**
 1. **Модуль питания MB102:** Подключается к внешнему USB 5V 2A+. Линия 5V с модуля MB102 идет *напрямую* на питание сервоприводов (VCC серво) и на силовую ногу усилителя MAX98357A (VIN). Общая земля (GND) MB102 и ESP32-S3 должна быть ****обязательно объединена****.
 2. **Конденсатор 1000 мкФ:** Устанавливается параллельно питанию сервоприводов на шину 5V и GND (соблюдая полярность). Он сглаживает броски тока при старте моторов. Второй конденсатор 1000 мкФ ставится по линии 5V платы ESP32.
 3. **Резистор 10 кОм:** Используется как Pull-up резистор на линию `RES` (Reset) дисплея к 3.3V для защиты от случайных наводок и сброса экрана при движении моторов.

2. Сетевая Архитектура и Протоколы Передачи Данных

Для минимизации задержек и исключения шизофрении нейросети при сборке проекта используется гибридная схема протоколов с отдельными каналами для Видео, Аудио и Управляющих JSON-команд.

2.1. Поток Video Передача Видео и Аудио

- **Видеопоток (ESP32 -> ПК):** HTTP MJPEG-стрим. ESP32 подготавливает буфер кадров **QVGA (320x240)** со скоростью 15 fps. Это дает минимальную нагрузку на процессоры ESP32 и идеально подходит для обработки алгоритмами OpenCV / YOLO на ноутбуке.
- **Аудио входящее (ESP32 -> ПК):** Поток PCM 16-bit 16kHz передается по бинарному **WebSocket-каналу** при срабатывании локального детектора громкости (VAD) или активном режиме разговора.
- **Аудио исходящее (ПК -> ESP32):** Сервер генерирует речевой поток через XTTS v2, сжимает его в формат OPUS/PCM и пересылает по WebSocket на ESP32 для прямой выгрузки в I2S усилитель MAX98357A.

2.2. Протокол Управления и Телеметрии

- **Канал Управления (Двунаправленный WebSocket):** Единое постоянно открытое WebSocket соединение `ws://[IP_LAPTOP]:8000/ws/robot``.
- **Формат Сообщений:** Строго валидируемый JSON.

```
{
  "target": "eyes",
  "action": "look_at",
  "params": {"x": 120, "y": 80},
  "servo": {"yaw": 105, "pitch": 80},
  "emotion": "curious"
}
```

3. Графика, Отрисовка Эмоций и Динамическое Зрение

Поскольку дисплей GMT130 (240x240, ST7789) не поддерживался в китайских бинарниках, на ESP32 поднимается собственный графический движок на базе библиотеки **LVGL 8.x** или прямой отрисовки Adafruit_GFX.

3.1. Генерация Анимированных Глаз(Procedural Procedural Eyes)

Рендеринг Зрачков: На экране 240x240 рисуются два скругленных прямоугольника или овала (глаза).

Координатная Связка Зрения: Когда Vision-модель на ПК обнаруживает объект (например, лицо человека на координатах кадра X_c, Y_c), сервер вычисляет нормализованные координаты смещения.

Интерполяция Глаз и Сервоприводов:

- Если смещение объекта небольшое — движутся только зрачки на экране в сторону объекта (по координатам X, Y).
- Если объект выходит за пределы центральной зоны — запускается плавный поворот сервопривода `Yaw` (GPIO 47) или `Pitch` (GPIO 21), возвращающий объект в центр кадра, а зрачки возвращаются в центральное положение относительно экрана.

Процедурное Моргание: Независимый таймер раз в 3-7 секунд производит сжатие глаз по оси Y до 2 пикселей на 150 мс, создавая «живой» эффект моргания.

4. Мозг Агента, Память (Obsidian + ChromaDB) и Логика Wake-Word

4.1. Стек Локального Сервера (MSI GP66)

- **Wake Word Detector:** `pyporcupine` (локальное слово «Джарвис» или «Робот»). Работает фоново. При детектировании переводит агента в **Активный Режим на 60 секунд**. Каждое новое реплицирование обнуляет таймер в 60 секунд.
- **STT (Speech-to-Text):** `Faster-Whisper` (модель `small` или `medium.en/ru` с запуском на TensorRT / CUDA).
- **LLM (Ядро мысли):** `Ollama` / `vLLM` с моделью `Qwen 2.5 7B Instruct` или `Llama 3.1 8B`.
- **Vision (Зрение):** `YOLOv8n` (30 fps, мгновенный поиск лиц и предметов) + `Moondream2` (Visual LLM для детального разбора неизвестных объектов).
- **TTS (Синтез Речи):** `Piper TTS` (быстрый, 50мс задержка) или `XTTS v2` (высокое качество голоса).

4.2. Архитектура Непрерывной Памяти в Obsidian и RAG

Для предотвращения рекурсий, «зацикливания» на старых фактах и переполнения контекста используется гибридная память:

А. Локальная База Векторов (ChromaDB)

Все Markdown-файлы из папки Obsidian автоматическая утилита на Python режет на чанки и индексирует в векторную БД **ChromaDB** с помощью эмбедингов `bge-m3` или `all-MiniLM-L6-v2`.

- При получении запроса от человека ИИ делает семантический поиск по ChromaDB.
- В контекст нейросети подтягиваются **только последние 3 релевантные записи**, а не вся история за год. Это полностью исключает «сход с ума».

Б. Иерархия Markdown Файлов

- `/Objects.md` — База известных предметов. Структура:
 - [Камера] #объект: Синяя кружка | Впервые замечен: 2026-09-14 | Хозяин пьет из нее кофе
- `/People.md` — Профили людей. Структура:
 - #человек: Артем | Должность: Создатель | Заметки: Любит собирать роботов
- `/Daily_Logs/YYYY-MM-DD.md` — Хроника событий дня.

5. Сценарии Поведения и Конечный Автомат (FSM)

5.1. Поведенческая Матрица Сценариев

Событие / Триггер	Реакция Зрения & Сервоприводов	Голосовая Реакция	Действие с БД / Obsidian
Человек вошел в комнату (YOLO заметил лицо)	Плавный поворот головы в сторону лица, зрачки фиксируются на человеке.	Если человека не было > 4 часов: «Привет! С возвращением. Как прошёл день?». Если < 4 часов: Молча	Запись в дневник дня: [НН:ММ] Человек вошел в поле зрения.

Событие / Триггер	Реакция Зрения & Сервоприводов	Голосовая Реакция	Действие с БД / Obsidian
		наблюдает, слегка кивает.	
Неизвестный предмет в кадре (YOLO видит неизвестный кластер)	Робот поворачивает камеру на объект, зрачки сужаются (режим «рассматривания»).	Агент может инициативно спросить: <i>«Я не узнаю этот предмет. Что это за штука?»</i>	После ответа человека идет создание записи в Objects.md с описанием от Vision-модели.
Никого нет в комнате > 2 часов	Голова опускается вниз (Pitch 45°). Подсветка дисплея плавно гаснет до 0% (GPIO 48 PWM). Сервоприводы отключают ШИМ (чтобы не гудели).	Полная тишина. Режим глубокого сна.	Запуск скрипта ночной оптимизации и очистки ChromaDB.
Произнесена Вейк-Фраза («Джарвис»)	Голова резко поднимается, зрачки расширяются, загорается подсветка экрана, радостный кивок.	Короткий звуковой сигнал (Bip) или реплика «Слушаю». Открывается окно диалога на 60 сек.	Фиксация в лог активности.

5.2. Резервный Режим (Ноутбук Выключен)

Если WebSocket-соединение с ноутбуком обрывается (таймаут 10 сек):

- ESP32 переключается в **Standalone Offline Mode** .
- Сервоприводы переходят в упрощенный алгоритм: при проигрывании любого звука сервопривод `Pitch` (GPIO 21) делает легкие покачивания вверх-вниз в такт с громкостью звука (простейшая имитация речи).
- На экран выводится статическая гифка/эмоция «Автономный режим».
- Запросы отправляются по Wi-Fi напрямую на внешние Cloud API (DeepSeek/Groq). Память Obsidian в этом режиме недоступна.

6. Модульность и Задел под Периферию (Робо-Рука)

Центральный сервер на ноутбуке выступает в роли **Оркестратора**. Робо-рука или другие модули собираются на отдельных ESP32 и подключаются к тому же WebSocket серверу (`ws://[IP_LAPTOP]:8000/ws/arm`).

Сценарий вызова: Человек говорит: *«Джарвис, подай отвертку»*.

Обработка ИИ: LLM на сервере определяет Tool Call: `{"tool": "arm_grab", "params": {"item": "screwdriver"}}`.

Исполнение: Сервер пересылает команду на ESP32 Робо-руки. Основной робот на ESP32-S3 CAM в этот момент поворачивает голову в сторону робо-руки и комментирует: *«Выполняю захват»*.

7. Готовый Системный Промпт для ИИ-Разработчика (Cursor IDE)

Скопируйте текст ниже и вставьте его в **Cursor IDE** (в окно системных правил `.cursorrules` или в качестве первого промпта при генерации кода).

Ты – ведущий Embedded и Python инженер. Твоя задача – написать полный программный комплекс для автономного робота AURA-Bot согласно следующей технической спецификации:

АРХИТЕКТУРА И СТЕК:

- ESP32-S3 N16R8 Firmware (C++ / PlatformIO, Arduino framework):
 - I2S Микрофон INMP441: SD=GPI042, WS=GPI01, SCK=GPI02.
 - I2S Усилитель MAX98357A: LRC=GPI014, BCLK=GPI013, DIN=GPI012.
 - Дисплей GMT130 IPS (ST7789, 240x240): SCL=GPI039, SDA=GPI038, RES=GPI040, DC=GPI041, BLK=GPI048. Движок процедурных глаз на LVGL/Adafruit_GFX (моргание, слежение за объектом).
 - Сервоприводы EMAX ES08MA II: Yaw=GPI047, Pitch=GPI021. Плавная интерполяция движений (Ease-in-out).
 - Видео: HTTP MJPEG stream (QVGA, 15fps) с камеры OV2640.

- Двухнаправленный WebSocket клиент для обмена JSON и аудио-потокм PCM 16kHz с сервером.

- Резервный режим: прямое переключение на Groq API при потере связи с сервером >10 сек.

2. Central Python Server (MSI GP66 Laptop):

- FastApi + WebSockets сервер управления.
- Wake Word: Porcupine ("Джарвис"). Активация сессии на 60 сек с автопродлением.
- STT: Faster-Whisper (CUDA).
- LLM: Ollama (Qwen 2.5 7B).
- Vision: YOLOv8 (отслеживание лиц/объектов + вычисление координат для сервоприводов) + Moondream2 (описание новых объектов).
- Память & RAG: Автоматическое чтение/запись в Obsidian Vault. Векторный индекс ChromaDB (bge-m3 эмбединги). Фильтрация релевантного контекста (не более 3 чанков), запись в Objects.md и Daily_Logs.
- Управление FSM (Бодрствование, Скука, Сон, Пробуждение, Режим неизвестного объекта).

Разбей код на модули, пиши чистый, отказоустойчивый C++ код для ESP32 и асинхронный Python 3.11+ код для сервера. Начни с реализации WebSocket протокола и видеострима.